

Semester Thesis

Image-Based Weight Estimation for Waste Items

Autumn Term 2022

Declaration of Originality

I hereby declare that the written work I have submitted entitled

Image-Based Weight Estimation of Waste Items

is original work which I alone have authored and which is written in my own words.¹

Author(s)

Nicola Burger

Student supervisor(s)

Fidel Esquivel Estay
Hendrik Kolvenbach

Committee members(s)

Supervising lecturer

Marco Hutter

With the signature I declare that I have been informed regarding normal academic citation rules and that I have read and understood the information on ‘Citation etiquette’ (<https://www.ethz.ch/content/dam/ethz/main/education/rechtliches-abschluesse/leistungskontrollen/plagiarism-citationetiquette.pdf>). The citation conventions usual to the discipline in question here have been respected.

The above written work may be tested electronically for plagiarism.

Place and date

Signature

¹Co-authored work: The signatures of all authors are required. Each signature attests to the originality of the entire piece of written work in its final form.

Intellectual Property Agreement

The student acted under the supervision of Prof. Hutter and contributed to research of his group. Research results of students outside the scope of an employment contract with ETH Zurich belong to the students themselves. The results of the student within the present thesis shall be exploited by ETH Zurich, possibly together with results of other contributors in the same field. To facilitate and to enable a common exploitation of all combined research results, the student hereby assigns his rights to the research results to ETH Zurich. In exchange, the student shall be treated like an employee of ETH Zurich with respect to any income generated due to the research results.

This agreement regulates the rights to the created research results.

1. Intellectual Property Rights

1. The student assigns his/her rights to the research results, including inventions and works protected by copyright, but not including his moral rights (“Urheberpersönlichkeitsrechte”), to ETH Zurich. Herewith, he cedes, in particular, all rights for commercial exploitations of research results to ETH Zurich. He is doing this voluntarily and with full awareness, in order to facilitate the commercial exploitation of the created Research Results. The student’s moral rights (“Urheberpersönlichkeitsrechte”) shall not be affected by this assignment.
2. In exchange, the student will be compensated by ETH Zurich in the case of income through the commercial exploitation of research results. Compensation will be made as if the student was an employee of ETH Zurich and according to the guidelines “Richtlinien für die wirtschaftliche Verwertung von Forschungsergebnissen der ETH Zürich”.
3. The student agrees to keep all research results confidential. This obligation to confidentiality shall persist until he or she is informed by ETH Zurich that the intellectual property rights to the research results have been protected through patent applications or other adequate measures or that no protection is sought, but not longer than 12 months after the collaborator has signed this agreement.
4. If a patent application is filed for an invention based on the research results, the student will duly provide all necessary signatures. He/she also agrees to be available whenever his aid is necessary in the course of the patent application process, e.g. to respond to questions of patent examiners or the like.

2. Settlement of Disagreements

Should disagreements arise out between the parties, the parties will make an effort to settle them between them in good faith. In case of failure of these agreements, Swiss Law shall be applied and the Courts of Zurich shall have exclusive jurisdiction.

Place and date

Signature

Contents

Preface	v
Abstract	vii
Symbols	ix
1 Introduction	1
1.1 Autonomous River Cleanup	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Requirements	2
2 State of the Art	5
2.1 Direct Methods	5
2.2 Indirect Methods	6
3 Methods	9
3.1 Class Average	9
3.2 Regression	9
3.2.1 2D Regression	11
3.2.2 3D Regression	11
3.2.3 Regression Models	15
3.3 Neural Network	17
4 Dataset	19
4.1 Requirements	19
4.2 Analysis	21
5 Results	25
5.1 Evaluation Metrics	25
5.2 Results	27
6 Discussion	31
7 Conclusion	33
Bibliography	36
A Calculations	37
A.1 Pixel to Squared Meter Conversion	37
B Datasheets	39

Preface

As a mechanical engineering student, I was fortunate to get the opportunity to pursue my semester thesis at the Autonomous River Cleanup (ARC). As a quick introduction, ARC is a student project at ETH Zurich that focuses on finding and developing new technologies to tackle riverine waste pollution. Their purpose, reputation as well as determined and open-minded team made it a no-brainer decision for me to apply to all of the interesting projects they had advertised. Hence, I am truly grateful for ARC to give me a shot at tackling the task of *Image-Based Weight Estimation for Waste Items* which was listed as my number one priority.

Hereby, I would also like to take the chance to thank my supervisors Fidel Esquivel Estay and Hendrik Kolvenbach! Thanks for giving me the opportunity to work on this truly unique and interesting project. Your support and coaching was invaluable and led to an exciting pursuit and exploration of different methods. Thank you!

Furthermore, a shoutout to all of the ARCies. It was a really fun experience and time at STB with you guys. Special thanks for all the help and inputs I received from Benjamin Sun, Dario Strübin and Xenia Voellmy which truly had an impact on this thesis.

Abstract

This semester project focuses on *Image-Based Weight Estimation for Waste Items*. Image-based weight estimation is an efficient method in waste management, allowing for faster processing and individual waste analysis. It also enables the generation of large scale waste statistics, crucial for tackling the increasing problem of riverine and oceanic waste pollution. For the pursuit of this task, we considered the following three image-based weight estimation methods: Class Average, Regression and Neural Networks. The simple Class Averaging method makes weight predictions based on an objects classification (being for instance glass or plastic). The Regression method additionally considers an objects estimated surface area leading to more accurate predictions. Finally, a Neural Network learns to predict an objects weight by being trained over a large quantity of examples. To validate and compare these methods, we created a dataset consisting of 325 unique waste items which provides a high variance of weights and object types. Finally, the results showed that using Regression leads to the highest accuracy of 96.43% of all predictions lying within half to double the ground truth weight of the respective items. This allows us to have a high certainty on our predictions while keeping the aforementioned tolerance band in mind.

Symbols

Symbols

u, v	image coordinate x, y [px]
f_x, f_y	focal length x, y [px]
c_x, c_y	principal point x, y [px]
K	camera intrinsics, $K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$

Acronyms and Abbreviations

AE	Absolute Error
ALE	Absolute Logarithmic Error
APE	Absolute Percentage Error
ARC	Autonomous River Cleanup
bbox	Bounding Box
ETH	Eidgenössische Technische Hochschule
IBWE	Image-Based Weight Estimation
ML	Machine Learning
PC	Pointcloud
SLE	Squared Logarithmic Error

Chapter 1

Introduction

As a brief introduction to the thesis, we discuss the **Autonomous River Cleanup** (ARC) project, the **Motivation** of the thesis, and the **Problem Description**.

1.1 Autonomous River Cleanup

ARC is a student-led project at ETH Zurich whose mission is to use technology, robotics and automation to counter riverine waste pollution. ARC can furthermore be viewed as a innovation hub where new solutions are discussed and developed, providing a platform for different approaches on how to tackle the problem of pollution in general. Currently, many ongoing projects tackle *Waste Retrieval / Sorting* and *Waste Analytics / Quantification*. The latter is where we are trying to develop a method to quantify waste by using image-based weight estimation techniques. For more information on the project, feel free to visit our [website](#).

1.2 Motivation

Weight estimation is an important task that stems from two distinct problems encountered in the industry as well as at ARC:

Industrial Applications For industrial applications, accurate weight estimation of waste items is a key task in waste management. It enables recycling facilities to optimize their processes which in turn leads to reduced costs and accurately charging customers for their recycled waste. Typically, the waste is weighed using scales using manual labor or for larger customers by weighing entire waste containers. An alternative and sought out method is to use computer vision algorithms to analyze the processed waste items and estimate their weight individually. This approach comes with the advantage of being faster and able to extract waste information of each individual object rather than a collection of waste items. The weight estimation accuracy however depends on the method and will never be as accurate as the traditional method of using a scale. Hence, the critical question is if the advantages outweigh the reduction in accuracy. The higher the accuracy, the more viable Image-Based Weight Estimation (IBWE) will be.

Waste Analytics For ARC, we encounter the problem of not having enough or inconsistent¹ data to determine the absolute amount of riverine waste. To act on

¹Studies vary largely. As an illustration of this problem, consider the following: Lebreton et al. [1], Meijer et al. [2], Schmidt et al. [3] report that 10 / 1000 / 10000 rivers respectively make up 90% of total riverine pollution, meaning either one or none of the studies got it right.

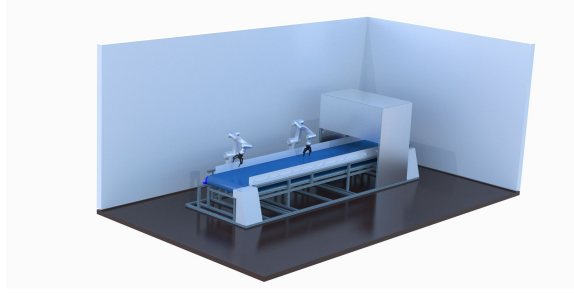


Figure 1.1: Sorting Stage

and receive funding to tackle waste pollution we must highlight the magnitude of the problem itself. Furthermore, if we know which rivers are most polluted, we could strategically place waste retrieval stages to counter pollution efficiently.

Due to the significance of the aforementioned problems, it becomes clear that there is a high demand for weight estimation solutions. Hence, we are motivated to find a method which allows us to make individual weight predictions on waste items.

1.3 Problem Statement

The problem at hand is to estimate the weight of waste objects passing along on a conveyor belt. This setup comes from the fact that recycling facilities process their waste by running it through multiple stages connected by conveyor belts as a method of transportation. In order to reproduce this process in a laboratory environment, ARC has developed a sorting stage for testing (see fig. 1.1). The setup includes a conveyor belt running at a maximum of 6.7cm/s , an RGB-D camera (RealsenseD455, see appendix B) and two robotic arms for object manipulation. The task of weight estimation shall be integrated into the already existing object detection pipeline with the option of expanding the sorting stage for custom modifications to aid the task at hand if required.

1.4 Requirements

Additionally, we have set the following requirements towards a viable solution that works for ARC:

(A) **Individual Predictions**

The solution shall be able to make per object predictions. This shall allow us to be flexible for future data analysis.

(B) **Easily Integratable**

Assuming the attendance of experienced ARC members, the solution shall be integratable into our waste retrieval systems as well as in recycling facilities within half a day.

(C) **Cheap**

The solution shall be cheap in order for it to be used in as many locations as possible.

(D) **Fast**

The solution shall be fast enough to keep up with the current throughput of

our conveyor belt system. Hence, we shall be able to make online predictions at a waste throughput speed of 6.7cm/s on a conveyor belt with a width of 70cm .

Chapter 2

State of the Art

To start off, let's quickly go over the physical characteristic of weight which is called mass and is defined as follows:

$$m = \rho * V \quad (2.1)$$

where:

$$\begin{aligned} \rho &= \text{density } \left[\frac{kg}{m^3} \right] \\ V &= \text{volume } [m^3] \end{aligned}$$

Weight estimation techniques are mostly based on either directly or indirectly estimating one or multiple of the physical properties in eq. (2.1). Direct, implying that there is physical interaction with an object such as for instance object manipulation. Hence, we divide these techniques into **direct** and **indirect methods**. Each of them come with their respective advantages and disadvantages which we need to consider when choosing a viable solution that satisfies our constraints.

2.1 Direct Methods

The most straightforward method is to simply use a scale. This method is considered to be a direct measurement technique where each object or a collection of objects is manually or automatically placed on a scale to be measured. In effect, a scale measures the gravitational force of an object using a load cell. The cells deformation leads to a change in electrical resistance, which causes a change in voltage that can be measured. The voltage can then be used to determine an objects weight. Needless to say that this technique is the most accurate since scales are a direct way to measure the weight of an object with an accuracy of up to one zeptogram ($10^{-24}g$) [4]. Unfortunately, this technique is not a compatible solution for our purposes due to the constraints we set ourselves. This can be seen when looking at two possible propositions. The first being to weigh each object individually. This would require robotic manipulation of the object with a robotic arm. This proposition fails three of our set requirements (not being easily integratable, cheap or fast). The second proposition would be to weigh a collection of sorted objects to determine the weight of said category. The second option might be more efficient and integratable, however, we would not be able to make individual predictions, which is insufficient with respect to our system constraints.

2.2 Indirect Methods

Indirect methods use computer vision to determine weight without physically interacting with the object. Most of the state of the art methods try to estimate the density and / or volume (see eq. (2.1)) to determine the mass of an object. Other methods are based on neural networks, which learn to make accurate predictions based on large training sets.

Density Estimation There are two methods to estimate the density of an object. The first is to use object detection, where we get each objects instance segmentation (including bounding box) and can classify an objects material (in our case e.g. plastic, glass, metal, etc.). Following fig. 2.1, we use the material and associate it with it's respective density from a material-density lookup table. An interesting alternative idea to estimate the density of an object is to use active thermography. Fig. 2.3 illustrates the process where a laser is pointed at an object. The targeted point on the object heats up due to the lasers energy. A thermal camera simultaneously captures the objects thermal diffusion. Because thermal diffusivity is inversely proportional to the objects density (see eq. (2.2)), the density can be estimated using various techniques, the best performing being to use a neural network [5].

$$\alpha = \frac{k}{\rho * c_p} \quad (2.2)$$

where:

$$\begin{aligned} k &= \text{thermal conductivity} \left[\frac{W}{m * K} \right] \\ \rho &= \text{density} \left[\frac{kg}{m^3} \right] \\ c_p &= \text{specific heat capacity} \left[\frac{J}{kg * K} \right] \end{aligned}$$

Volume Estimation Knowing the approximate scale or volume of an object often suffices to make a fairly accurate prediction of an objects weight given the object type. Since the object type is very specific, the weight can be estimated using regression algorithms. This approach is commonly found in agriculture to make weight estimations on live stock or food such as discussed in the following papers [6, 7, 8, 9, 10, 11]. Typically, object detection is used to extract the segmentation or bounding box area which represents the scale of an object (see fig. 2.2, in green). A more accurate estimation is to combine the instance segmentation with pointcloud (PC) information to apply surface reconstruction of an object using either photogrammy or depth information (see fig. 2.2, in blue). Lastly, there is the option to extract an objects volume using convolutional neural networks [12].

Neural Networks Due to the rapid advance in research and capabilities in deep learning, a lot of computer vision tasks can be done using convolution neural networks. Deep learning research in the field of weight estimation has not been as extensive, however, there have been two impactfull papers published on the topic. The first paper, image2mass [12], is a weight estimation pipeline based on two neural networks that estimate the density and volume of an object. These networks have an Xception architecture and are trained in parallel. Backpropagation is done using the loss of the multiplied outputs of the two networks. What distinguishes this pipeline from most is that it is set up to make predictions on a collection of various household objects rather than just one object type. In the second paper, Tafuro et al. [13], various PC- and graph-based neural networks are compared on their ability to predict the weight of strawberries. They use an RGB-D images for object detection and PC extraction. The strawberries PC is then filtered and

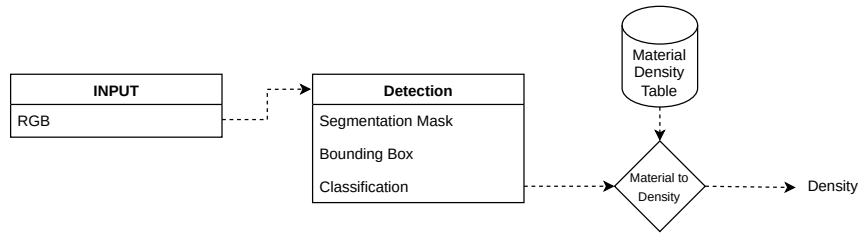


Figure 2.1: Density Estimation using Object Classification

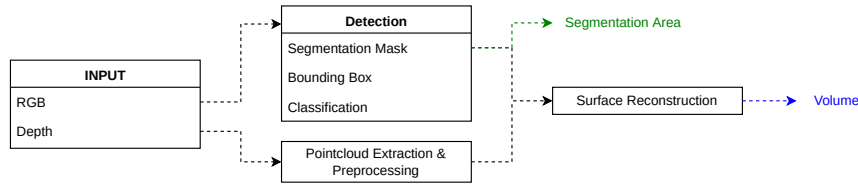


Figure 2.2: Volume Estimation using Instance Segmentation (green) and Surface Reconstruction (red)

sampled to be used as an input to the neural networks. They also used a Random Forest model with simplified features extracted from the RGB image and PC. The results showed that the simpler Random Forest model outperformed the more complex neural networks. It is argued that the simpler model generalizes better and that the neural networks would most likely need a vastly larger training set to achieve similar performance.

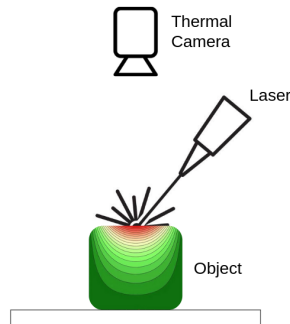


Figure 2.3: Density Estimation using Active Thermography

Chapter 3

Methods

Given the decision of using an indirect, image-based weight estimation technique, we analyzed which methods would be most suitable for the task at hand. In the following subchapters we explain the chosen techniques: **Class Average**, **2D Regression**, **3D Regression** and **NN**.

3.1 Class Average

This method computes the average weight of all items belonging to the same category / class (see eq. (3.1)). Once this is done for each class of objects, weight predictions can be made by predicting each object's weight as the class mean. This method can also be implemented using the median weight of classes (see eq. (3.2)). It is the most straight forward of all discussed methods and is used as a benchmark for evaluating the performance of other methods.

$$m_{class} = \frac{1}{n} \sum_{i=1}^n m[i] \quad (3.1)$$

$$m_{class} = \begin{cases} m[\frac{n+1}{2}], & \text{if } n \text{ is odd.} \\ \frac{m[\frac{n}{2}] + m[\frac{n}{2}+1]}{2}, & \text{if } n \text{ is even.} \end{cases} \quad (3.2)$$

where:

$m[i]$ = mass of i^{th} class object
 n = number of class objects

ARC already has predefined waste classes, which are: PET Bottle, Plastic Bottle, Glass, Metals, Paper / Carton, Tetrapak. Therefore, we make use of them in order to find the class averages based on our dataset and expand on one more class labeled as Wrappers / Foils. fig. 4.2 shows the distribution of weights per class and illustrates the varying class averages and medians.

3.2 Regression

As discussed in State of the Art, knowing the size or volume of an object often suffices to make accurate weight predictions. For our use case, we can make use of the RGB-D image stream provided by the sorting stages camera (see fig. 3.1 and fig. 3.2). This allows for either two- or three-dimensional image analysis, which is why this section is divided into **2D Regression** and **3D Regression**.



Figure 3.1: RGB Camera Feed Input

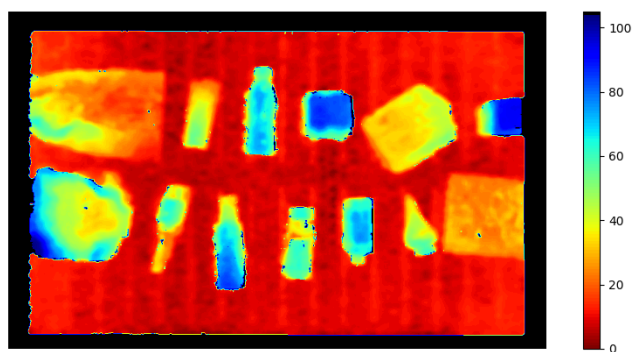


Figure 3.2: Depth Camera Feed Input

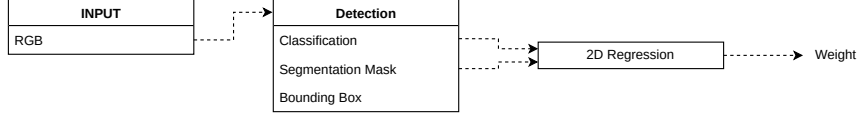


Figure 3.3: 2D Regression Pipeline

3.2.1 2D Regression

The 2D Regression method is illustrated in fig. 3.3. An RGB image is used as an input to a object detection module. This in turn predicts the objects instance segmentations as well as classifies each instance. These predictions are used as an input to a regression model which provides weight estimations for each object. The model can be trained using the classification and segmentation ground truths. To illustrate the viability of this process, we extract the ground truth annotations of all images in the dataset and plot the segmentation area to the weight of each object whilst considering its class (see fig. 3.4). A clear correlation between the two variables can be seen. These correlations can be captured using the right regression models to get accurate weight estimations. As a sidenote: The segmentation area of an image is given in pixels. This means that the segmentation area depends on distance between the camera and conveyor belt which can vary depending on the setup. Hence, to be flexible with the setup, it is necessary to first convert the pixel units to the uniform unit of *mm* (see appendix A.1).

3.2.2 3D Regression

The 3D Regression method is illustrated in fig. 3.5. The inputs to the pipeline are RGB and depth images. The RGB image is used as an input to the object detection module which predicts the objects instance segmentation and category. Rather than in 2D Regression, the predicted segmentation mask is not used directly as an input to the regression model. Moreover, the depth image is converted to a PC and preprocessed in the Scene Preprocessing module. The Mask Scene module then uses the segmentation mask of an object in combination with the scene PC to get an objects PC. The object PC is then used as an input to the Surface Reconstruction module to estimate the total surface area of an object. It is important to note that for our use case, we are not interested in the enclosed volume of an object but rather the surface area. This is due to the fact that the objects we analyze are waste items which are packaging materials in most cases. This means that the content is neglected and therefore the surface area is more meaningful than the volume enclosed by an object. Finally, the estimated surface area in combination with the object classification is used as an input for the regression model which predicts the weight of an object. The newly discussed modules are discussed in the following paragraphs:

- (A) Scene Preprocessing
- (B) Mask Scene
- (C) Surface Reconstruction

Scene Preprocessing Scene preprocessing describes the process of converting a depth image into a PC which is preprocessed to contain only the objects PC information. This step contains the following operations: Depth-to-PC and Conveyor Belt Removal.



Figure 3.4: Segmentation area plotted against the corresponding objects weight

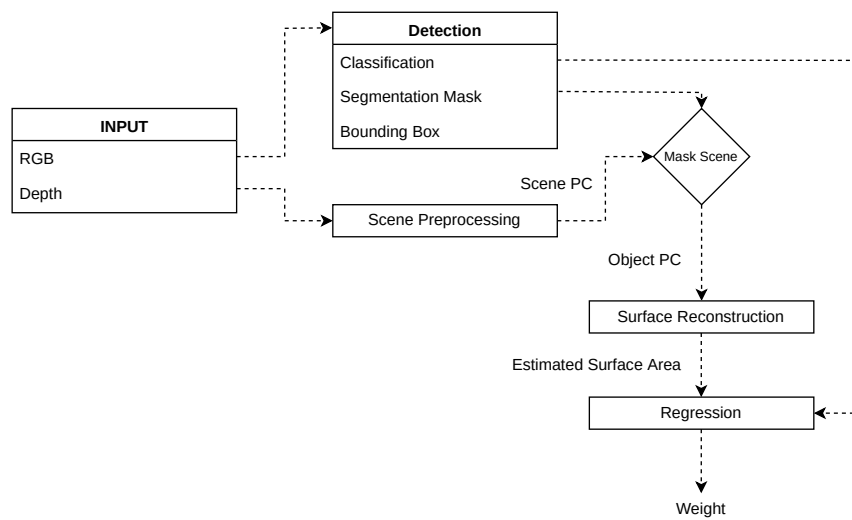


Figure 3.5: 3D Regression Pipeline

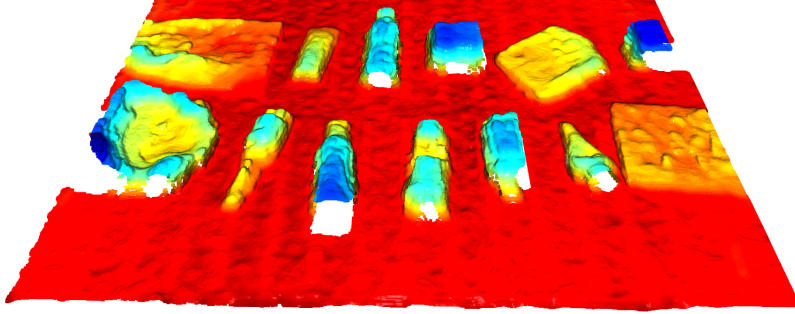


Figure 3.6: Depth-to-Pointcloud Operation

(A) Depth-to-PC

Using the camera intrinsics, it is possible to convert a depth image to a PC. The operation computes the PC coordinates according to eq. (3.3), eq. (3.4) and eq. (3.5) as described in [14]. fig. 3.6 illustrates the resulting PC following this operation.

$$z = \text{depth}(u, v) \quad (3.3)$$

$$x = \frac{(v - c_x) * z}{f_x} \quad (3.4)$$

$$y = \frac{(u - c_y) * z}{f_y} \quad (3.5)$$

(B) Conveyor Belt Removal

The entire Scene Preprocessing module is done in order to further reconstruct the surface of each objects PC. Hence, we do not require the conveyor belt PCs. In the contrary, having the conveyor belt points within an objects point-cloud would result in a bias during the Surface Reconstruction. Hence, it is advantageous to remove the conveyor belt points. There are multiple ways of doing this. One method would be to measure the camera mounting height to the conveyor belt and remove all points that are farther away than said distance. The negative aspects of this method are that we require a measurement for each setup and more importantly, if the camera is not oriented perpendicular to the conveyor belt, the operation might cut important points that should not be removed. Another method is to use RANSAC to estimate the conveyor belt planes with respect to the cameras frame of reference. After adding a padding threshold, we remove all points below the computed plane. This is the preferable method and used in the scene preprocessing pipeline. fig. 3.7 illustrates the result of applying the RANSAC conveyor belt removal operation.

Mask Scene In order to extract an individual objects PC, we apply the corresponding instance segmentation mask upon the scene PC. As an example, an objects PC is illustrated in fig. 3.8

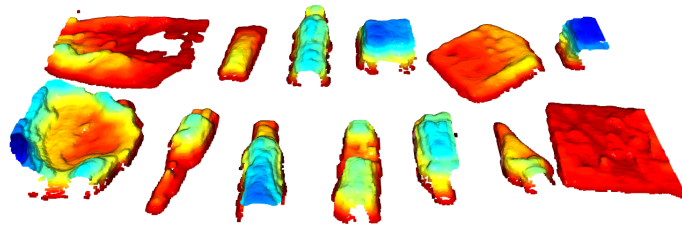


Figure 3.7: Conveyor Belt Removal using RANSAC plane estimation

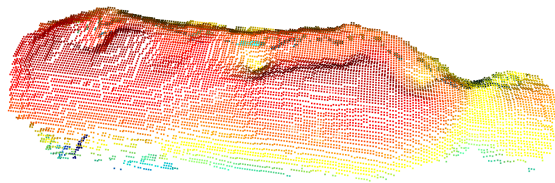


Figure 3.8: Object PC by applying the objects mask on the scene PC

Surface Reconstruction To get our surface area estimate, we now apply Surface Reconstruction upon an objects PC. Surface Reconstruction is a well studied field in PC manipulation. For our specific use-case, we considered the following three popular and suiting surface reconstruction algorithms for the task at hand:

(A) **Alpha Shape**

The Alpha Shape algorithm creates faces based on an imaginary sphere placed through the PC. The sphere is arranged such that exactly three points intersect it. Wherever there is no other point within this sphere of influence, the three points are connected and a surface is created. The radius of said sphere is fixed and set by the *alpha* parameter. Hence, the alpha shape algorithm results in a set of faces that reconstruct the surface of an object. More information on the implementation can be found in Fischer [15]. An example of the algorithms result is shown in fig. 3.9.

(B) **Ball Pivoting**

The Ball Pivoting algorithm uses the concept of a ball rolling along the surface of a PC. Imagine a ball with a fixed radius falling onto a PC. The ball will be absorbed by three points. A face will then be constructed between these points. The ball will then continue to roll along the PC in all directions. Whenever it is absorbed by another set of three points, the process repeats. To counter the ball falling into crevices of the PC, this algorithm is repeated iteratively with increasing sizes of ball radiuses. More information on the implementation can be found in Bernardini et al. [16]. An example of the algorithms result is shown in fig. 3.10.

(C) **Poisson**

The Poisson algorithm computes an indicator function which is used to obtain a watertight surface by extracting an appropriate isosurface. To do so, it solves the following standard Poisson problem:

$$\Delta X = \nabla * \nabla X = \nabla * \vec{V} \quad (3.6)$$

where:

$$\begin{aligned} X &= \text{Indicator Function} \\ \vec{V} &= \text{Oriented Points} \end{aligned}$$

The computed indicator function X is a scalar field where all points within an object have a value of 1, whilst the exterior points get the value of 0 assigned. The solution to the problem is then given by computing an isosurface from said scalar field. More information on the implementation can be found in Kazhdan et al. [17]. An example of the algorithms result is shown in fig. 3.11.

3.2.3 Regression Models

In ML, there are typically two types of problems which are either a classification or regression task. The most known regression task being linear regression. The following are the most common and suitable regression models for the task at hand. Furthermore, our feature vector consists of the surface area (estimated or regressed). We use a polynomial feature expansion to expand the feature up to the third degree (see eq. (3.7)).

- Linear Regression [18]
- Ridge Regression [19]

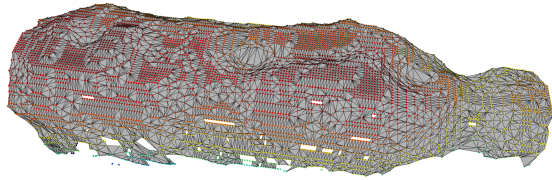


Figure 3.9: Surface Reconstruction using Alpha Shape

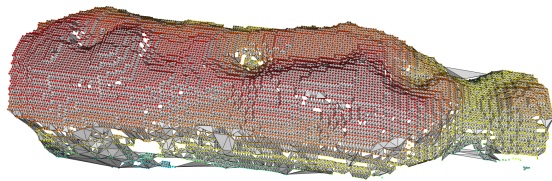


Figure 3.10: Surface Reconstruction using Ball Pivoting

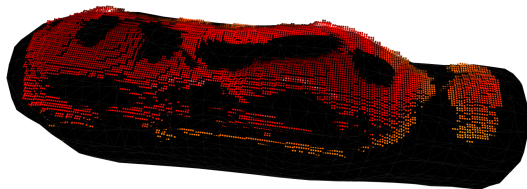


Figure 3.11: Surface Reconstruction using Poisson Algorithm

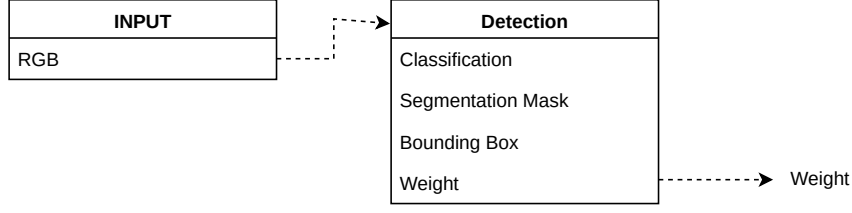


Figure 3.12: NN Regression Pipeline

- Lasso Regression [20]
- Elasticnet [21]
- Bayesian Ridge Regression [22]
- Random Forest Regression [23]
- Support Vector Regression [24]
- Linear Support Vector Regression [25]

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_m & x_m^2 \end{bmatrix} \quad (3.7)$$

The performance of these models depends on the specific task. Since they are not computationally expensive, we evaluated all of them for comparison.

3.3 Neural Network

The field of deep learning has developed exponentially through the past decade. Hence, there are a variety of different types of NN models. For our task, we ended up deciding to expand on a preexisting object detection network. The selected model and implementation depends on the MMDetection library [26]. Because this is an object detection network, the output layer of the model contains two separate tasks and losses. In order to make the network additionally predict weight, we either add a direct (see fig. 3.13) or indirect (see fig. 3.14) regression head to the output layer. The difference between the two being that the direct regression head is a linear layer between the output layer and the weight prediction node, whilst for the direct regression head one or multiple linear layers are placed between the two. Hence, the model now has three losses which are summed up by their respective weights (see eq. (3.8)). The bounding box (bbox) loss is an L1 loss, the class prediction loss is a cross entropy loss and the weight prediction loss is an absolute logarithmic error loss.

$$L = w_{bbox} * L_{bbox} + w_{classification} * L_{classification} + w_{weight} * L_{weight} \quad (3.8)$$

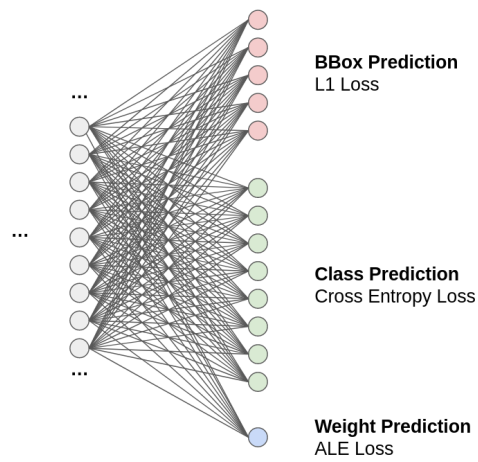


Figure 3.13: NN Output Layers, direct weight estimation

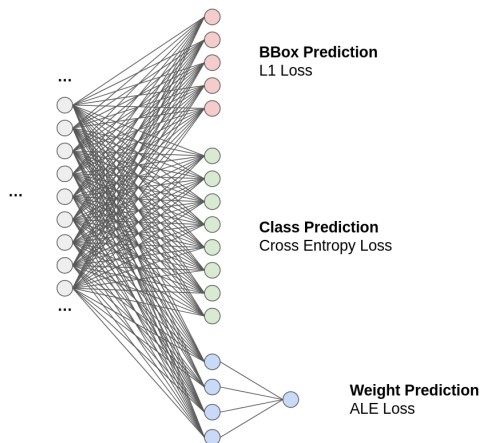


Figure 3.14: NN Output Layers, indirect weight estimation

Chapter 4

Dataset

ARC already has set up a couple of datasets which we were able to use for prototyping and testing the discussed methods. However, we realized that there were three problems for clearly evaluating these methods on those datasets. The first being that the datasets were unbalanced with respect to the object categories and the second that the variance of weights within these categories was not high enough. This most likely happened due to the fact that the waste objects were not randomly sampled from garbage collections. The third problem was that there was only a training and test split, but no validation split. Hence, we would not be able to know when the NN was overfitting whilst training. Therefore, we created a new dataset with the intention of eliminating these problems by collecting random trash bags of ARC team members and splitting the objects into a train, validation and test split.

4.1 Requirements

In order to create a new and viable dataset, we set ourselves the requirements of **Waste Categorization, Dataset Size, Split Sizes, Features**.

Waste Categorization ARC already has a clear and sensible waste categorization which was used for the new dataset. We did however expand the categorization by a new class for Wrapper / Foils. Hence, our waste categorization requirement is to classify all objects into either of these classes. In fig. 4.1, we provide a couple of objects for each category.

- PET Bottle
- Plastic Bottle
- Glass
- Metals
- Paper / Carton
- Tetrapak
- Wrapper / Foils



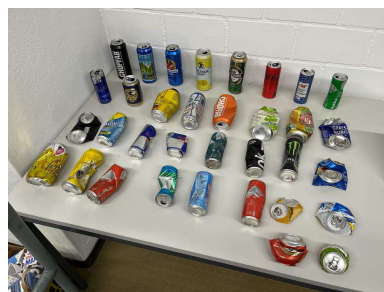
(a) PET Bottles



(b) Plastic Bottles



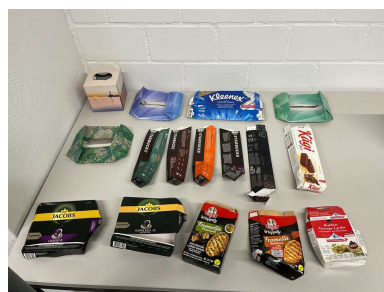
(c) Glass



(d) Metals



(e) Tetrapak



(f) Paper / Carton



(g) Wrappers / Foils

Figure 4.1: Example Objects by Category

Dataset Size In order to evaluate our models, we would like to have a high variance of different objects and their respective weights. Therefore, we decided to use a minimum of 50 objects per category, which is substantially more than in the previous datasets where we had a maximum of 20 objects per category. Furthermore, whilst keeping in mind that we would be training a NN, we decided to have exactly the same number of objects per category in order to balance the dataset. Due to the fact that there were not too many objects, we ended up with the minimum set number of 50 objects per category.

Split Sizes As mentioned previously, we require a validation split to determine when the NN starts to overfit. Normally, a dataset is split up into training, validation and test sets with 70%, 10% and 20% respectively. However, due to the fact that we do not have a lot of objects, we require our test set to be larger. We decided that 20 objects per category would satisfy a high variance of objects within our test set and that we would most likely require at least 10 objects per category for validation purposes. Hence, to maximize our training split with respect to the previous constraints, we ended up with the following split sizes shown in table 4.1.

Table 4.1: Split Sizes

Split	Size [%]	Objects per Category
Training	40	20
Validation	20	10
Test	40	20
Total	100	50

Features The features chosen were quite straight forward and based on the task at hand. Hence, the obvious two features that were certainly required are the object category and the weight of an object. Object classification requires no further explanation, however, the weight of a waste object requires further details. In most cases, waste objects are some sort of packaging. What we observed especially for PET Bottles, Plastic Bottles and Glass objects was that they contained left-over content. Take as an example a half-empty PET bottle containing water. Weighing the bottle including the water would strongly bias our models towards a higher weight estimate for PET Bottles. It is clear that this does not make sense and therefore requires us to first empty all waste objects before weighing them. Furthermore, what we found during testing the methods on the previous datasets was that the depth images would result in large distortions when objects were transparent. This happened especially for the PET Bottles, Plastic Bottles and Glass categories. Hence, we decided to add a transparency feature for future analysis of such objects.

4.2 Analysis

To get an overview of the dataset, we quickly go over the following visualizations: **Objects, Weight Distribution, Categorical Data, Image Resolution and Dataset Size.**

Objects In total, the dataset consists of 325 objects. fig. 4.1 illustrates the types of objects used for the dataset with respect to the various categories.

Weight Distribution To get an understanding of the weight distribution, we use a boxplot visualization and distinguish between categories in fig. 4.2. As seen in the illustration, each category has a clear weight bandwidth. What sticks out is that the heaviest class is Glass, whilst the lightest category are Wrappers / Foils (with objects weighing only 1g).

Categorical Data As mentioned previously, we get transparency issues within our depth images for some categories. This is visualized in fig. 4.3, where we find that the categories to watch out for are: PET Bottle, Plastic Bottle, Glass and the newly added Wrapper / Foils.

Image Resolution The realsense D455 (see appendix B) camera is used as our image stream input. The camera-conveyorbelt distance is set at $0.68m$. The placement turns out to create a trade-off between resolution and depth-range. The closer we position the camera to the conveyor belt, the higher a resolution we obtain. However, at the same time, the depth image range will decrease due to the infrared sensors, whose region of interception decreases. Due to other tasks at ARC, we strategically placed the camera at the mentioned $0.68m$ distance to the conveyor belt. This results in a 1280×800 resolution for the RGB image and a 1080×720 resolution for the depth image. Examples of each can be found in fig. 3.1 and fig. 3.2.

Dataset Size The dataset was done by randomly mixing and throwing all of the objects of a certain split onto the conveyor belt. This process was repeated three times per split. Hence, each object is annotated at least three times. To view the size of the dataset, we look at the absolute number of unique waste objects and the corresponding annotations throughout. Furthermore, we provide the number of images that were annotated per split. These values can be seen in table 4.2.

Table 4.2: Dataset Size

Split	Objects	Images	Annotations
Training	130	36	447
Validation	65	17	208
Test	130	36	458
Total	325	89	1113

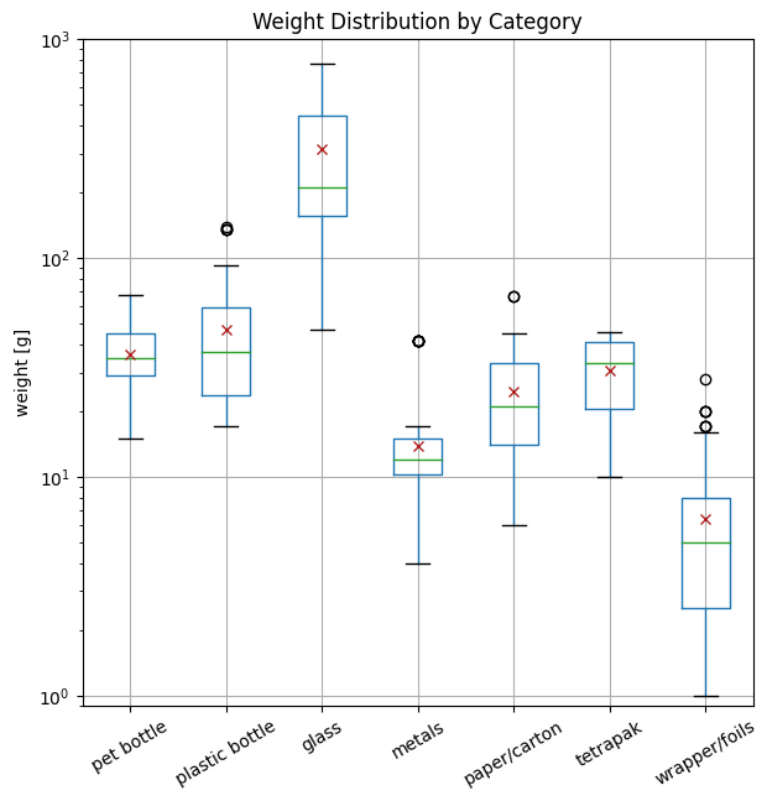


Figure 4.2: Weight Distribution by Category (red: mean, green: median)

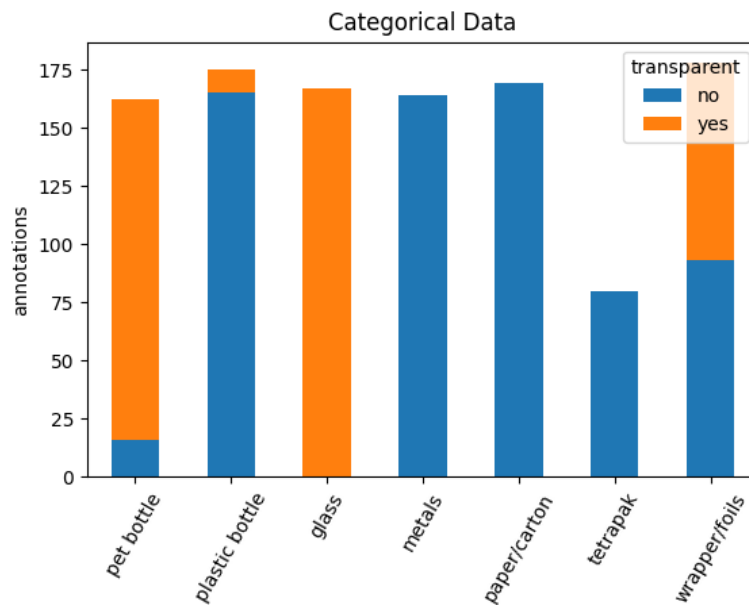


Figure 4.3: Categorical Data w.r.t. Transparency and Annotations

Chapter 5

Results

To evaluate and compare the discussed methods in chapter 3 we need to have a meaningful evaluation metric. Furthermore, we have to run all of the methods with the same conditions and obtain the results of each used technique. These two tasks are discussed in the following subchapters: **Evaluation Metrics** and **Results**.

5.1 Evaluation Metrics

For regression tasks, such as the one we are working on, there is a large variety of evaluation metrics to consider. These metrics are related to a specific loss that can be computed by comparing the weight predictions with the ground truths. In order to find out which one fits best for our use-case, we must first establish constraints as to which methods we should consider.

Requirements Consider the following situation: We have two objects. Object 1 weighs $10g$ whilst object 2 weighs $50g$. How do we evaluate predictions made on these two objects, compared with each other? The following steps illustrate why the final requirements were chosen:

(A) **Absolute Error**

For regression tasks, we typically use the Absolute Error (AE) or Squared Absolute Error (SAE). For simplicity, the AE is implemented and compared between the two objects. In fig. 5.1a we can see the respective error curves of the objects predictions. What immediately stands out is that the error metric is not proportional. This means that heavier objects get an equally high error if the prediction is off by the same amount. In fig. 5.1a we can see this by looking at the red points. Both objects were predicted to be $10g$ heavier than they actually were. However, object 2 is a lot heavier than object 1 and should hence be punished less and proportionally to its weight, not absolutely. This observation results in our first requirement: the evaluation metric must be *proportional*.

(B) **Absolute Percentage Error**

In the following, we analyze the Absolute Percentage Error (APE) which has the property of proportionality. The error curves of this metric with the previous example is illustrated in fig. 5.1b. Here, we can see that proportionality is provided, however, if both objects are predicted to have a very low weight, both are penalized the same (visualized with the red dot in fig. 5.1b).

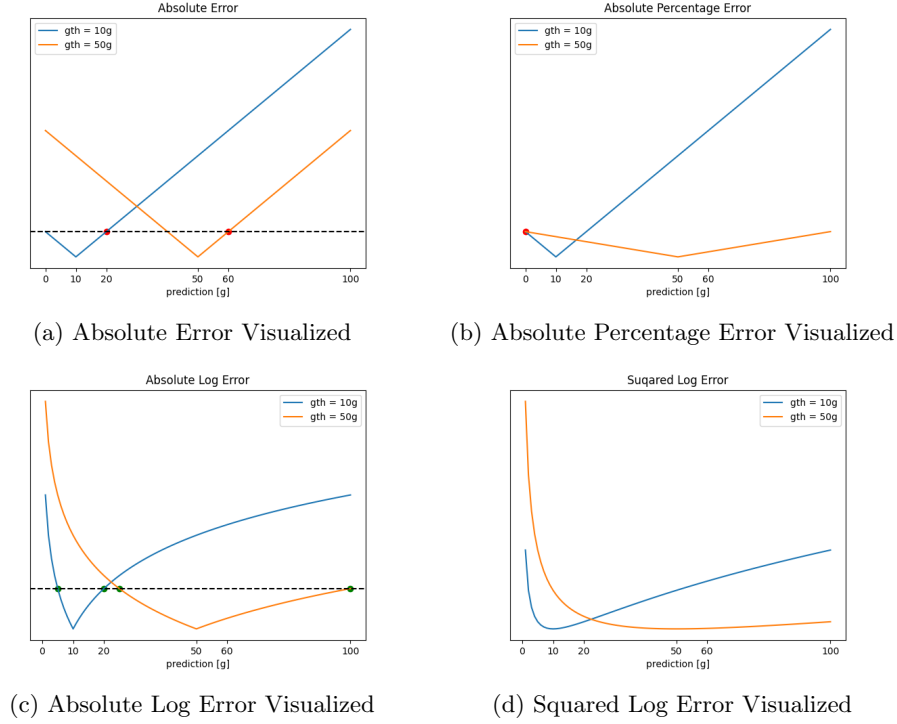


Figure 5.1: Evaluation Metrics Visualized by Comparing the Predictions of two Objects

This property is undesirable, since we want the penalty to be higher for object 2. Therefore, we set a further requirement which is that the error metric must be *logarithmically proportional*. This additional property rectifies the aforementioned problem.

(C) Absolute Log Error

The Absolute Log Error (ALE) metric is *logarithmically proportional* (and therefore implicitly also *proportional*). The ALE curve of the example is visualized in fig. 5.1c. It is important to note that the base of the logarithm does not have any affect on the curve itself, except for its scale. The ALE seems to satisfy all our needs for a useful error metric for evaluating weight estimations. An additional positive feature that can be observed is that if an object is estimated to weigh half or double its ground truth value, the error will always be the same, regardless of the objects ground truth weight (visualized as green dots in fig. 5.1c).

Due to the discussed example, we end up with two constraints for our evaluation metric. It must be *proportional* and *logarithmically proportional*. The latter implicitly implies the existence of the first. Hence, our requirement for the evaluation metric is to be *logarithmically proportional*.

Evaluation Metric Going over a collection of possible error metrics used for regression tasks (see table 5.1), we find that we are left with two viable solutions: ALE and Squared Log Error (SLE). Comparing the two, we find that the squared log error (see fig. 5.1d) suppresses small errors whilst highly penalizing larger errors. This fact is not negative, however, too heavy of a penalization for bad estimates.

ALE does a better job of still recognizing small errors, which is why we decided it would be the better option for our task. For future reference, the ALE is defined by eq. (5.1)¹. The base of the logarithms does not matter, since the errors will just be scaled by a factor. There is a use for choosing the correct base for our use case which is determined in the next section.

$$ALE(t, p) = |\log_a(t + 1) - \log_a(p + 1)| \quad (5.1)$$

where:

t = ground truth weight
 p = predicted weight

ALE-below-1 Error Bound Our chosen evaluation metric comes with a nice feature that can be used for evaluating our predictions. This being that the ALE has the property that all predictions that are either half or double of what the ground truth value is will result in the same error (seen by the horizontal line indicated in fig. 5.1c). This range is referred to as the ALE-below-1 error bound, which is illustrated in fig. 5.2. By choosing the base a of eq. (5.1) to be 2, the error will end up to be 1. This number acts as a threshold to indicate if a prediction falls between the error bound of being within half or double the ground truth of an objects ground truth weight (see eq. (5.2)). Therefore, our evaluation metric is defined by eq. (5.3).

$$p \in \text{ALE-below-1} \leftrightarrow p \in [\frac{t}{2}, 2t] \quad (5.2)$$

$$ALE(t, p) = |\log_2(t + 1) - \log_2(p + 1)| \quad (5.3)$$

Table 5.1: Split Sizes

Error Metric	Logarithmically Proportional
Min Ratio Error	no
Absolute Error	no
Absolute Log Error	yes
Absolute Percentage Error	no
Squared Error	no
Squared Error +	no
Squared Error ++	no
Squared Log Error	yes
Huber Loss	no
Huber Loss + Quantile Loss	no
Log-Cosh Loss	no
Quantile Loss	no

5.2 Results

The following results were obtained by running the discussed methods on our dataset. The evaluation is based on the ALE metric and makes use of the ALE-below-1 error bound. The results show all of the methods side-by-side and are

¹The usage of adding 1 to the prediction and ground truth value is done to bypass numerical errors when encountering values of 0

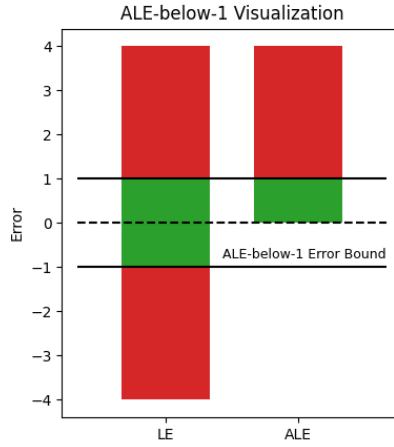


Figure 5.2: ALE-below-1 Error Bound (green: fulfilled, red: outside of boundary)

split by one level of depth. For the first experiment, we look at the results over all of the object categories and for the second we look at the performance of each method based on a specific category of objects. This shall allow us to find out which methods work best under which circumstances.

Cumulative ALE Every method estimates the weight of each annotation in the dataset. The number of errors that fall below a certain threshold can be counted in order to find the percentage of estimates that were within a specific error bound. The progression of thresholds and number of corresponding predictions below said thresholds are visualized in fig. 5.3, where the verticle blue line indicates the ALE-below-1 error bound.

Results by Category Because we have different types of objects, some methods might work better for glass objects, whilst others may perform better on metals. Therefore, we conduct an experiment based on the object categories. Since we are specifically interested in the fraction of predictions that fall within the ALE-below-1 error bound, we compare all of the methods with respect to the object categories by the ALE-below-1 fraction of predictions in table 5.3.

Ground Truth vs Object Detection To run the Class Average and Regression tasks, we used the annotated ground truths of the dataset. This shall provide results which do not rely on the performance of any object detection module which might bias the results. However, since the NN model we are using is built upon the object detection module and can not be fed the absolute ground truths, the results are bound to be worse for the NN method. Hence, it might appear that the NN method is inferior to the other methods, yet, the results have to be compared with consideration of said fact.

Computational Complexity Class Averaging and 2D Regression do not lead to any computational overhead, since all of the features are already given by the Neural Network. The Neural Network method is just an additional linear layer which has no recognizable impact on the computational time required for the operation. However, 3D Regression requires a large amount of preprocessing of the PC (see fig. 3.5), leading to large overheads. These can be seen in table 5.2. When considering

an average of 12.5 objects per scene² and using the fastest surface reconstruction method which is Alpha Shape, we would require 11.7 seconds per scene³. With a conveyor belt speed of 6.7cm per second, the objects would already have traveled 78.4cm before the next weight estimations could take place. This would be enough to estimate the weight of each object 1-2 times whilst running by on a conveyor belt and therefore satisfies our constraints.

Table 5.2: Surface Reconstruction Complexity

Operation	Avg. Execution Time [ms]
Scene Preprocessing	507.3
Masking Scene	5.2
Object Preprocessing	512.8
Surface Reconstruction: Alpha Shape	378.4
Surface Reconstruction: Ball Pivoting	1057.8
Surface Reconstruction: Poisson Algorithm	660.4

Table 5.3: Dataset Size

ALE-below-1	Mean	Median	Area	Alpha Shape	Ball Pivoting	Poisson	NN
PET Bottle	100	100	100	100	100	100	67.21
Plastic Bottle	94.74	85.96	100	100	100	96.49	66.67
Glass	75.44	40.35	100	100	100	92.97	5.26
Metals	81.48	85.19	92.59	94.44	94.44	94.44	27.78
Paper / Carton	72.22	64.81	87.04	81.48	83.33	72.22	51.85
Tetrapak	76.92	76.92	100	96.15	96.15	100	57.69
Wrapper / Foils	70.59	64.71	100	84.31	92.17	72.55	5.89
Total	82.50	74.17	96.43	93.80	93.80	89.82	40.37

²This is the average of our created dataset. Unfortunately, we are unaware of the industry standards.

³The experiments were run with an Intel Core i7, 64-bit system in Python. There may be a significant speed increase experienced when integrating the same approach with e.g. C++

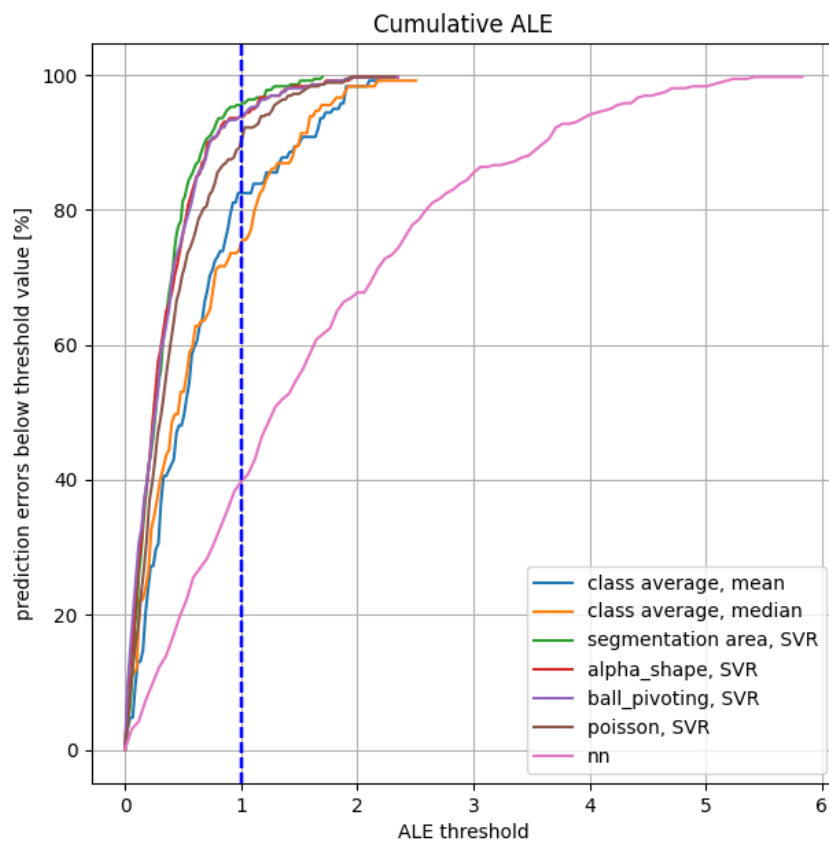


Figure 5.3: Cumulative ALE shown for all methods

Chapter 6

Discussion

The results display a solid performance of the methods, except for the NN. The individual methods are discussed in the following and finally compared.

Class Average With an ALE-below-1 of 82.50% / 74.17% for the mean / median methods, the class average methods performs astonishingly well considering its simplicity. Overall, the mean slightly outperforms the median. This increase in performance is due to the mean resulting in smaller ALE over all the objects because it represents the overall object classes better than the median. This can be seen clearly when looking at the weight distribution (see fig. 4.2). The only class that is represented better by the median are the metals. This is due to a couple of outliers which drastically increase the mean due to their abnormally high weights. Aside of such cases, we can confidently say that the using the mean works better than using the median.

Regression The 2D as well as 3D regression methods were run for all of the regression models discussed in the Regression Models chapter. Out of all the models, the Support Vector Regression model [27] performed best for all classes and methods. It does not penalize objects which lie within a specified error bound around the regression. Hence, it captures as many datapoints that surround the ground truth weights, but ignores small variations. In comparison to other methods such as linear regression, SVR puts less weight on outliers and more weight on capturing a clear trend within the data. What stands out for the regression methods is that the 2D regression method slightly outperforms the 3D regression methods with a ALE-below-1 of 96.43%. At first, this may appear counterintuitive because the 3D regression method computes the surface rather than just segmentation area and hence should have preciser information about the object. It was observed that this occurs because the objects PCs are captured only from one angle. So an object that lies at the border of an image will result in a PC capturing its top and side. However, if the same object lies directly under the camera, only the top side of the object will be extracted to a PC. This results in varying surface area estimations for the same object, whilst the segmentation area will always have a very similar value. Hence, due to the inconsistency of computed surface areas for the same objects, the 3D regression model performs worse than the 2D regression model. This could be countered if we would have tracked and archived an objects PC throughout its journey on the conveyor belt. This would allow us to capture the object from more angles and use PC stitching to get a more complete PC of the objects.

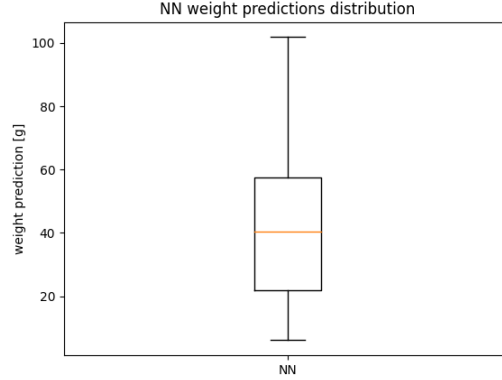


Figure 6.1: NN Weight Predictions Distribution

NN With a ALE-below-1 of 40.37%, the NN performed significantly inferior to the previously discussed methods. However, we can not compare the results of the NN with the other methods directly since it did not get the luxury of using the classification and segmentation ground truths. Furthermore, we were only able to implement and perform limited tuning due to time constraints. Analyzing the models predictions, we find that the model seems to try a shortcut by arbitrarily making predictions in a specific range of values as can be seen in fig. 6.1. We find the predictions lying around a mean / median of $41.69g$ / $40.42g$ with a standard deviation of $22.40g$. We assume that the model was not trained well enough and with more hyperparameter tuning, might have refrained from making arbitrary estimations around the aforementioned range.

Comparison & Improvements As an overall comparison between the methods, we find that the 2D Regression method performs best, with a ALE-below-1 of 96.43%. With that being said, we believe that improving the 3D Regression method by adding PC stitching might prove to bypass the 3D Regression method in performance because of the additional information. Furthermore, the NN could potentially be improved which would allow an end-to-end weight estimation pipeline. All in all we recommend the 2D regression task to be suitable for ARC due to its performance, being computationally inexpensive and the currently best performing method with a ALE-below-1 of 96.43%.

Chapter 7

Conclusion

Concluding this thesis, we analyzed state-of-the art weight estimation methods which could be used for estimating the weight of waste items for ARC. Class Average, 2D Regression, 3D Regression and a NN were chosen to be best suitable for the task at hand. To evaluate and test the proposed methods, we created a new dataset with a large variety of 325 unique waste objects across 7 waste categories. The methods were trained on a training set consisting of 60% of all 1113 annotations. The rest of the dataset was used for testing purposes, which allowed us to compare the different methods. The highest performing of which is 2D Regression with an ALE-below-1 of 96.43%, outperforming the 3D Regression, Class Average and NN methods (with an ALE-below-1 of 93.80%, 82.50% and 40.37% respectively). Hence, we have found a suitable and well performing weight estimation method for ARC. This shall allow ARC to gather and collect an additional piece of information for waste analytics and quantification. Furthermore, for gaining a higher weight estimation accuracy, we propose improving the 3D Regression method by using PC Tracking and Stitching of an object to uniformly capture an objects PC and gain more accurate information about its surface area.

Bibliography

- [1] L. C. Lebreton, J. V. D. Zwet, J. W. Damsteeg, B. Slat, A. Andrady, and J. Reisser, “River plastic emissions to the world’s oceans,” *Nature Communications*, vol. 8, 6 2017.
- [2] L. J. J. Meijer, T. V. Emmerik, R. V. D. Ent, C. Schmidt, and L. Lebreton, “More than 1000 rivers account for 80 into the ocean,” 2021.
- [3] C. Schmidt, T. Krauth, and S. Wagner, “Export of plastic debris by rivers into the sea,” *Environmental Science and Technology*, vol. 51, pp. 12 246–12 253, 11 2017.
- [4] X. F. Li, G. J. Tang, Z. B. Shen, and K. Y. Lee, “Resonance frequency and mass identification of zeptogram-scale nanosensor based on the nonlocal beam theory,” *Ultrasonics*, vol. 55, pp. 75–84, 2015.
- [5] T. Aujeszkzy, G. Korres, M. Eid, and F. Khorrami, “Estimating weight of unknown objects using active thermography,” *Robotics*, vol. 8, 12 2019.
- [6] C. M. Sabliov, D. Boldor, K. M. Keener, and B. E. Farkas, “Image processing method to determine surface area and volume of axi-symmetric agricultural products,” *International Journal of Food Properties*, vol. 5, pp. 641–653, 2002.
- [7] E. Samperio, I. Lidón, R. Rebollar, M. Castejón-Limas, and C. Álvarez Aparicio, “Lambs’ live weight estimation using 3d images,” *Animal*, vol. 15, 5 2021.
- [8] K. Jun, S. J. Kim, and H. W. Ji, “Estimating pig weights from images without constraint on posture and illumination,” *Computers and Electronics in Agriculture*, vol. 153, pp. 169–176, 10 2018.
- [9] H. Hassannejad, G. Matrella, P. Ciampolini, I. D. Munari, M. Mordonini, and S. Cagnoni, “A new approach to image-based estimation of food volume,” *Algorithms*, vol. 10, 6 2017.
- [10] M. Kashiha, C. Bahr, S. Ott, C. P. Moons, T. A. Niewold, F. O. Ödberg, and D. Berckmans, “Automatic weight estimation of individual pigs using image analysis,” *Computers and Electronics in Agriculture*, vol. 107, pp. 38–44, 2014.
- [11] M. O. Balaban, G. F. Ünal Şengör, M. G. Soriano, and E. G. Ruiz, “Using image analysis to predict the weight of alaskan salmon of different species,” *Journal of Food Science*, vol. 75, 4 2010.
- [12] T. Standley, O. Sener, D. Chen, and S. Savarese, “image2mass: Estimating the mass of an object from its image,” 2017.
- [13] A. Tafuro, A. Adewumi, S. Parsa, G. E. Amir, and B. Debnath, “Strawberry picking point localization ripeness and weight estimation.” Institute of Electrical and Electronics Engineers Inc., 2022, pp. 2295–2302.

-
- [14] “Estimate point clouds from depth images in python — by chayma zatout — better programming.”
 - [15] K. Fischer, “Introduction to alpha shapes.”
 - [16] F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, G. Taubin, and S. Member, “The ball-pivoting algorithm for surface reconstruction.”
 - [17] M. Kazhdan, M. Bolitho, and H. Hoppe, “Poisson surface reconstruction,” 2006.
 - [18] N. S. Foong, C. Y. Ming, C. P. Eng, and N. K. Shien, “An insight of linear regression analysis.”
 - [19] D. W. Marquardt and R. D. Snee, “Ridge regression in practice,” *American Statistician*, vol. 29, pp. 3–20, 1975.
 - [20] “What is lasso regression definition, examples and techniques.”
 - [21] “Elastic-net regularization in learning theory - sciencedirect.”
 - [22] “Introduction to bayesian linear regression — by will koehrsen — towards data science.”
 - [23] “Random forest regression. a basic explanation and use case in 7... — by nima beheshti — towards data science.”
 - [24] “Support vector regression.”
 - [25] Q. Klopfenstein and S. Vaiter, “Linear support vector regression with linear constraints,” *Machine Learning*, vol. 110, pp. 1939–1974, 7 2021.
 - [26] K. Chen, J. Wang, J. Pang, Y. Cao, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu, J. Xu, Z. Zhang, D. Cheng, C. Zhu, T. Cheng, Q. Zhao, B. Li, X. Lu, R. Zhu, Y. Wu, J. Dai, J. Wang, J. Shi, W. Ouyang, C. C. Loy, and D. Lin, “Mmdetection: Open mmlab detection toolbox and benchmark,” 6 2019.
 - [27] “An introduction to support vector regression (svr) — by tom sharp — towards data science.”

Appendix A

Calculations

A.1 Pixel to Squared Meter Conversion

To convert from pixels to mm^2 we can look at the relation between the pixels on the projection plane with respect to the pixel projected onto the conveyor belt (see figure A.1). Geometrically, we get to the relationship displayed in equation A.1. The focal length in x direction is defined as A.2 which allows us to factor out the focal length as seen in A.3. The same applies for the focal length in y direction. Hence we can get b as seen in equation A.4.

This allows us to compute a conversion factor from pixels to squared milimeters (see equation A.5). Finally, this allows us to multiply a conversion factor onto the original segmentation area x to get y (see equation A.6). As a sidenote, we define a pixels width / height as $\sqrt{px}$. We also directly convert all to milimeters instead of meters due to the size of normal waste items.

$$\frac{d}{b} = \frac{f}{a} \quad (A.1)$$

$$f_x = \frac{f * \sqrt{px}}{a} \quad (A.2)$$

$$\frac{d}{b} = \frac{f}{a} = \frac{f_x}{\sqrt{px}} \quad (A.3)$$

$$b = \frac{d * \sqrt{px}}{f_x} \quad (A.4)$$

$$c(d) = \frac{d^2 * px}{f_x * f_y} * \frac{1}{px} \quad (A.5)$$

$$y = x * c(d) \quad (A.6)$$

where:

- x = segmentation area $[px]$
- y = segmentation area $[mm^2]$
- f_x = focal length in x direction $[\sqrt{px}]$
- f_y = focal length in y direction $[\sqrt{px}]$
- f = focal length $[m]$
- a = 1 pixel on projection plane in milimeters $[mm]$
- b = distance on conveyor belt $[mm]$
- $c(d)$ = pixel to squared meter conversion factor $[\frac{mm^2}{pixel}]$
- d = distance from camera to conveyor belt $[mm]$

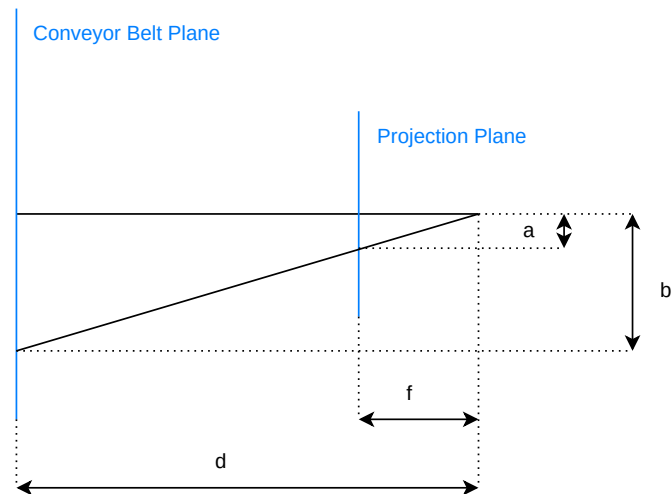


Figure A.1: Simplified Camera-Conveyor-Belt Setup

Appendix B

Datasheets

Intel® RealSense™ Depth Family Offerings

	D415	D435/D435i	D455	SR305	L515
Use Case	3D scanning, facial authentication, robotics: collision avoidance and pick and place, recognition and interaction use cases	3D scanning, facial authentication, robotics: collision avoidance and pick and place, recognition and interaction use cases	3D scanning, robotics: collision avoidance, recognition and interaction use cases	Indoor short range objects with small details, Facial Recognition, gesture (hand tracking), close in scanning	Cases where scanning accuracy is paramount and lighting is controlled. Box measurement, object, and room scanning
Value	Multicamera in same FOV; Best priced stereo camera	Global shutter with wide FOV; Multicamera in same FOV	Global shutter with wide FOV; Multicamera in same FOV; Match FOV; Global shutter RGB with depth and HW sync trigger	Coded light solution with good accuracy at short range; Excellent price	High Accuracy over entire range; world's smallest hi-res LiDAR camera
Optimal Range (Stereo camera can see further out - Distance - accuracy worsens geometrically)	.4m to 3m	.4m to 3m	.4m to 6m	.2m to 1.5m (accuracy is -consistent)	.4m to 9m (accuracy is -consistent)
Z Error < 2% Range and Min Z @ Max Resolution	<2% at 2m Min Z ~50 cm	<2% at 2m Min Z ~30cm	<2% at 4m Min Z ~40cm	<3% at 1.5m Min Z ~20cm	~3mm to ~16mm thru 9m Min Z ~25cm
Environment	Indoor & Outdoor	Indoor & Outdoor	Indoor & Outdoor	Indoors	Indoors
Max Depth Resolution	1280x720 @ 30fps 848x480 @ 90fps	1280x720 @ 30fps 848x480 @ 90fps	1280x720 @ 30fps 848x480 @ 90fps	640x480 @ 60fps	1024x768 @ 30fps
Depth FOV HD/Shutter¹	64°x41°/Rolling	86°x57°/Global	86°x57°/Global	65°x50°/Global	70°x55°
RGB/Shutter¹	2MP/64°x41°/Rolling	2MP/64°x41°/Rolling	1MP (matched to depth FOV)/Global	2MP/Rolling	2MP/Rolling
Product Dimensions (mm)	99x23x20	90x25x25	124x29x26	139x26x12	61 diameter x 26
IMU	NO	NO (D435i YES)	YES	NO	YES

¹ FOV is measured +/-3° of stated value

* Final technical specs not available until QS for each product

